

Las reducciones

Comentario: Ningún ejercicio reviste mayor dificultad (hay que pensar un poco más en los ejercicios 3 a 5, pero se recomienda no dejar de considerarlos).

Ejercicio 1. Considerando la reducción de HP a LU descrita en clase, responder:

a. Explicar por qué la función identidad, es decir la función que a toda cadena le asigna la misma cadena, no es una reducción de HP a LU.

Conceptualmente:

- La función de reducción de HP a L_U implica cambiar los qR de la MT U a qA. Si usamos la función identidad, terminamos obteniendo exactamente el mismo resultado que la MT U original, seguimos resolviendo el problema general de aceptación, pero no el Halting Problem.

Formalmente:

Para que una reducción sea válida, se tiene que cumplir que
 $w \in HP \Leftrightarrow f(w) \in L_U$

Si usamos la función identidad, esto no se cumple.

Supongamos:

- Máquina M que se detiene en qR para un w.
- Entonces $w \in HP$ (porque M paró, lo de adelante de la flecha la cumplimos)
- Pero $f(w) \notin L_U$ (porque M no aceptó, rechazó. Rompemos la $\Leftrightarrow$)

b. Explicar por qué las MT M2 generadas en los pares de salida ($\langle M2 \rangle, w$), o bien paran aceptando, o bien loopean.

Por la modificación que le hacemos al código.

Para reducir HP a L_U , la función f “modifica el código” de MT U, para crear MT HP:

1. si MT U iba a qA, MT HP da qA
2. si MT U iba a qR, MT HP da qA
3. si MT U loopea, MT HP loopea.

Al eliminar el estado qR del código de MT U, le quitamos la capacidad de rechazar.

c. Explicar por qué la función utilizada para reducir HP a LU también sirve para reducir HP^c a LU^c .

Si f es una reducción de A y B , por definición:

$$x \in A \Leftrightarrow f(x) \in B$$

por lógica proposicional (el contrareciproco), eso es exactamente lo mismo que decir:

$$x \notin A \Leftrightarrow f(x) \notin B$$

Lo que es equivalente a:

$$x \in A^c \Leftrightarrow f(x) \in B^c$$

d. Explicar por qué la función utilizada para reducir HP a LU no sirve para reducir LU a HP.

Condición de correctitud:

- La misma establece “Asigna a toda cadena w que pertenece al lenguaje $L1$ una cadena $f(w)$ que pertenece a $L2$. Y a toda cadena w que no pertenece a $L1$, una cadena $f(w)$ que no pertenece a $L2$ ”
- Para que la condición sea válida, si $(\langle M1 \rangle, w) \notin L_U$, entonces $f(\langle M1 \rangle, w) \notin HP$

Por qué no sirve:

- Si $M1$ acepta $\rightarrow M2_{HP}$ acepta: esto es está bien.
- Si $M1$ rechaza w ($(\langle M1 \rangle, w) \notin L_U$) $\rightarrow M2_{HP}$ acepta ($(\langle M2 \rangle, w) \in HP$): MAL

Ahí violamos la condición de correctitud:

Se mapeó un elemento que NO estaba en el L_U a uno que SÍ está en el HP.

La reducción no preserva la estructura del problema.

e. Explicar por qué la siguiente MT M_f no computa una reducción de HP a LU: dada una cadena válida $\langle M \rangle, w$, M_f ejecuta M sobre w , si M acepta entonces genera la salida $\langle M \rangle, w$, y si M rechaza entonces genera la cadena 1.

La función f no es total computable, ergo no cumple la Condición de computabilidad de las reducciones.

El error acá está en que f se pone a EJECUTAR la M que le viene como parametro. Si esa M no frena nunca, f se queda loopeando, y nunca llega a generar ni probar M_2 .

La función de reducción podría “ejecutar” al M que le viene SOLO si nos aseguran que está en R y va a parar. Si no nos dicen nada, la función de reducción debe solo encargarse de ser traductora de sintaxis.

Ejercicio 2. Sabiendo que $LU \in RE$ y $LU^c \in CO-RE$:

a. Probamos en clase que existe una reducción de LU a L_{Σ^*} . En base a esto, ¿qué se puede afirmar con respecto a la ubicación de L_{Σ^*} en la jerarquía de la computabilidad?

Nota:

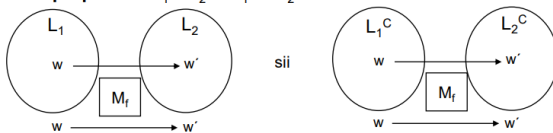
- ¿Qué representa LU ? Que hay una máquina MT M que acepta a un w
- ¿Qué representa L_{Σ^*} ? Que hay una máquina que acepta SIEMPRE TODO y no rechaza nada

En primer lugar:

- Sabemos por enunciado que $L_U \leq L_{\Sigma^*}$
- Al saber que existe una reducción de L_U a L_{Σ^*} sabemos que L_{Σ^*} es igual o más complejo que L_U
- Además nos dicen que $L_U \in RE$, y como L_{Σ^*} es igual o más complejo que L_U , como R es menos complejo que $RE \rightarrow$ **Primera conclusión: $L_{\Sigma^*} \notin R$**

En segundo lugar, usando la propiedad

- Otra propiedad: $L_1 \leq L_2$ sii $L_1^c \leq L_2^c$. La reducción es la misma.



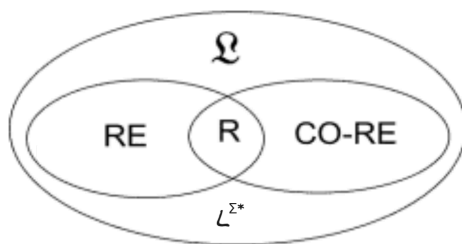
- Por esa propiedad, podemos decir que $L_U \leq L_{\Sigma^*} \rightarrow L_U^c \leq L_{\Sigma^*}^c$
- Si $L_U^c \notin RE$, entonces $L_{\Sigma^*}^c \notin RE$
- Y como $L_U^c \leq L_{\Sigma^*}$, y $L_U^c \notin RE \rightarrow$ **segunda conclusión $L_{\Sigma^*} \notin RE$**

En tercer lugar:

- Como el $L_{\Sigma^*}^c \notin RE$, entonces $\rightarrow$ **tercera conclusión $L_{\Sigma^*} \notin CO-RE$**

Ergo, dado que:

- $L_{\Sigma^*} \notin R$
- $L_{\Sigma^*} \notin RE$
- $L_{\Sigma^*} \notin CO-RE$
- Entonces:
 - **$L_{\Sigma^*} \in L - (RE \cup CO-RE)$**



b. Se prueba que existe una reducción de L_U^c a $L_\emptyset$. En base a esto, ¿qué se puede afirmar con respecto a la ubicación de $L_\emptyset$ en la jerarquía de la computabilidad?

Primero:

- Sabiendo que $L_U^c \notin RE$
- y dado que existe una reducción de L_U^c a $L_\emptyset$
- Entonces $L_\emptyset$ tiene que ser o más igual difícil que L_U^c . $\rightarrow L_\emptyset \notin RE$.

En segundo lugar:

- $L_\emptyset^c \in RE$
(Es el lenguaje de las máquinas que aceptan al menos una cadena. Este lenguaje sí es RE)
- Si el **complemento** de $L_\emptyset$ pertenece a RE entonces $\rightarrow L_\emptyset \in CO-RE$
(podés reconocer si una máquina NO pertenece a $L_\emptyset$, es decir, si acepta algo).

Ejercicio 3. Sea el lenguaje $D_{HP} = \{w_i \mid M_i \text{ para a partir de } w_i\}$ (considerar el orden canónico). Encontrar una reducción de D_{HP} a HP.

Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud.

- **Lenguajes:**
 - **D_{HP} :** representa al conjunto de MTs del lenguaje diagonal, tal que $PARAN$ con su la cadena que comparte índice como input. La máquina M_5 debe aceptar w_5 (orden canonico, por ejemplo, para alfabeto $\{0, 1\}$, sería $\{B, 0, 1, 00, 01, \dots\}$ Si M_5 para con '01', $M_5 \in D_{HP}$).
 - **HP:** representa el conjunto de MT que paran (ya sea que acepta o no acepta).
- **Pregunta:** Si agarro la cadena que está en la posición i de la lista y se la doy como entrada a la máquina que ocupa esa misma posición i , ¿esa máquina frena?
- **Idea general de la reducción:**
 - Para la entrada W_i de D_{HP} , generar la $\langle M_i \rangle$ (en el orden canónico).
 - Pasarle como entrada el par $\langle M_i, W_i \rangle$ a HP
- **¿F es total computable?**
 - Dado que el conjunto de todas las Máquinas de Turing es enumerable, existe un algoritmo que puede generar la $\langle M_i \rangle$ a partir de su índice i en tiempo finito. Es total porque está definida para cualquier entrada del dominio, y es computable porque la creación de la máquina se da siempre en un tiempo finito.
- **¿f es correcta?**

Condiciones de correctitud: Asigna a toda cadena que w perteneciente a $L1$ una $f(w)$ perteneciente a $L2$, y a toda cadena que no pertenece a $L1$ una $f(w)$ que no pertenece a $L2$.

 - $W_i \in D_{HP} \rightarrow M_i \text{ se detiene a partir de } w \rightarrow f(w_i) = (\langle M_i \rangle, w_i) \in HP$
 - $W_i \notin D_{HP} \rightarrow M_i \text{ no se detiene a partir de } w \rightarrow f(w_i) = (\langle M_i \rangle, w_i) \notin HP$

Ejercicio 4. Sean TAUT y NOSAT los lenguajes de las fórmulas booleanas sin cuantificadores tautológicas (satisfactibles por todas las asignaciones de valores de verdad) e insatisfactibles (insatisfactibles por todas las asignaciones de valores de verdad).

Encontrar una reducción de TAUT a NOSAT.

Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud.

- **Lenguajes:**
 - **TAUT:** Representa al conjunto de fórmulas booleanas que son siempre verdaderas para cualquier asignación de sus variables (tautologías).
 - **NOSAT:** Representa al conjunto de fórmulas booleanas que son siempre falsas para cualquier asignación de sus variables (contradicciones o insatisfactibles).
- **Pregunta:** Si tengo una fórmula lógica, ¿existe alguna forma de transformar el problema de saber si es "siempre verdad" en un problema de saber si algo es "siempre mentira"?
- **Idea general de la reducción:**
 - Para la entrada ϕ de TAUT, generar la fórmula $\neg\phi$ (negar la fórmula original).
 - Pasarle como entrada la fórmula resultante $\neg\phi$ a MT NOSAT.]
- **¿f es total computable?**
 - Dado que ϕ es una cadena de caracteres que representa una fórmula, existe una MT capaz de leer dicha cadena y simplemente escribir el símbolo de negación $\neg$ al comienzo de la misma.
Es total porque está definida para cualquier fórmula del dominio y es computable porque la operación de concatenar un símbolo es muy sencilla, evidentemente se realiza siempre en tiempo finito.
- **¿f es correcta?**
 - $\phi \in \text{TAUT} \rightarrow \phi$ es siempre verdadera $\rightarrow \neg\phi$ es siempre falsa $= f(\phi) \rightarrow \neg\phi \in \text{NOSAT}$
 - $\phi \notin \text{TAUT} \rightarrow \phi$ no es siempre verdadera $\rightarrow \neg\phi$ no es siempre falsa $= f(\phi) \rightarrow \neg\phi \notin \text{NOSAT}$

Ejercicio 5. Se prueba que existe una reducción de L_U^c a L_{Σ^*} (y así, como $L_U^c \notin RE$, entonces se cumple que $L_{\Sigma^*} \notin RE$).

La reducción es la siguiente.

Para toda w : $f(\langle M1 \rangle, w) = \langle M2 \rangle$, tal que $M2$, a partir de su entrada v , ejecuta $|v|$ pasos de $M1$ a partir de w , y acepta si $M1$ no acepta.

Probar que la función definida es efectivamente una reducción de L_U^c a L_{Σ^*} .

Comentario: hay que probar su total computabilidad y correctitud.

CONTEXTO:

- **Lenguajes:**
 - L_U^c : Representa el conjunto de pares $\langle M1 \rangle, w \rangle$ tales que la máquina M_1 RECHAZA la cadena w .
 - L_{Σ^*} : Representa al conjunto de MT que aceptan absolutamente todas las cadenas del alfabeto Σ^* .
- **Pregunta:** ¿Cómo podemos usar $M2$ (que reconoce máquinas que aceptan todo) para saber si una máquina específica se RECHAZA con una entrada particular?
- **Idea general de la reducción:**
 - Para la entrada $\langle \langle M1 \rangle, w \rangle$, generamos el código de una nueva máquina $M2$.
 - Esta $M2$, al recibir una cadena cualquiera de v , simula $M1$ sobre w , pero solo durante $|v|$ pasos.
 - Si al cabo esa cant de pasos $M1$ aún no aceptó, entonces $M2$ ACEPTA. Si $M1$ acepta dentro de ese límite, $M2$ RECHAZA.

LO QUE NOS PIDEN: ¿Es eso una función de reducción válida? Para que una función de reducción sea válida, debe ser total computable y cumplir la condición de correctitud.

- **¿f es total computable?**

- Dado que la función M_f solo debe definir el comportamiento (digamos, “escribir el código”) de una M_2 , que recibe un w , calcula $|v|$, y simula M_1 durante $|v|$ pasos, y aceptar solo si M_1 no aceptó en esos $|v|$ pasos.
- Escribir un código finito y simular a otra máquina por una cantidad finita de pasos puede ser realizado por una MT que siempre se va a detener.
- Por tanto, f es total computable porque existe una MT M_f que puede calcular $f(\langle M_1 \rangle, w)$ para cualquier entrada y siempre se detiene.

- **¿f es correcta?**

Condiciones de correctitud: Asigna a toda cadena que w perteneciente a L_1 una $f(w)$ perteneciente a L_2 , y a toda cadena que no pertenece a L_1 una $f(w)$ que no pertenece a L_2 .

$(\langle M_1 \rangle, w) \in L_1^c \rightarrow M_1 \text{ no acepta } w \text{ en } |v| \text{ pasos} \rightarrow f(w) = M_2 \text{ acepta} \rightarrow (\langle M_1 \rangle, w) \in L_2^*$

$(\langle M_1 \rangle, w) \notin L_1^c \rightarrow M_1 \text{ acepta } w \text{ en } |v| \text{ pasos} \rightarrow f(w) = M_2 \text{ rechaza} \rightarrow (\langle M_1 \rangle, w) \notin L_2^*$